

Bericht für die Betriebspraxisphase 2

Betriebspraxisphase vom 01.07.2023 – 30.09.2023

Semesterarbeit

Fachhochschule Vorarlberg

Elektrotechnik Dual, Jahrgang 2021

Betreut von

Patrick Ritschel und Michael Woog

Vorgelegt von

Michael Geuze

Dornbirn, 04.09.2023

Inhaltsverzeichnis

Darstellungsverzeichnis	4
Literaturverzeichnis	5
Abkürzungsverzeichnis	6
1 Praxisbericht der beiden Kompetenzbereiche	7
1.1 Dispopapier	7
2 Lernziele	9
2.1 Elektronik	9
2.2 Informatik	9
3 Aufgabenstellung	10
3.1 CI/CD Phase 2	10
3.2 ETP-Testadapter	10
3.3 Vertiefung Embedded Programmierkenntnisse	10
4 Erwartete Ergebnisse	12
4.1 CI/CD Phase 2	12
4.2 ETP-Testadapter	12
4.3 Vertiefung Embedded Programmierkenntnisse	12
5 Umsetzung	13
5.1 CI/CD Phase 2	13
5.1.1 Tools	14
5.1.2 CI/CD Standardprozedur	14
5.2 Erstellung eines generischen Docker Containers für STM32 Controller	16
5.2.1 Vorgehen	16
5.3 Erstellung eines Docker Containers zum ändern der Testadapter Software	18
5.4 ETP-Testadapter	19
5.5 Vertiefung Embedded Programmierkenntnisse	20
5.5.1 Ansteuerung des Mikrocontroller Clock Output	20
5.5.2 Konfiguration der einzelnen Clock Prescaler und Multiplexer	21
5.5.3 Konfiguration des System Tick Timer	23
5.5.4 Erstellung eines Moduls für den LCD 1602 Punkt-Matrix-Bildschirm	23

5.5.5	Erstellung eines Protokolls für den DHT11 Feuchtigkeitssensor	24
6	Gegenüberstellung der erwarteten Ergebnisse und des Erreichten - Zusammenhang Studieninhalte und Praxisinhalte	26
7	Resümee der Betriebspraxisphase: Ausgeführte Tätigkeiten, Lernzielerreichung, persönliche Erfahrungen – Reflexion und Selbsteinschätzung	27
8	Zusammenfassung des Lerntagebuchs	29
8.1	Zusammenfassung KW27	29
8.2	Zusammenfassung KW28	29
8.3	Zusammenfassung KW29	29
8.4	Zusammenfassung KW30	30
8.5	Zusammenfassung KW31	30
8.6	Zusammenfassung KW32	31
8.7	Zusammenfassung KW33	31
8.8	Zusammenfassung KW34	32
8.9	Zusammenfassung KW35	32
8.10	Zusammenfassung KW36	33
9	Zusammenfassung der Abwesenheiten	34
	Anhang	35

Darstellungsverzeichnis

Abbildung 1: Aufbau der CI/CD Infrastruktur	13
Abbildung 2: PlatformIO verwendet den falschen Slash.....	18
Abbildung 3: Aufbau der Pin Ein- und Ausgangsstufe [3, p. 160].....	21
Abbildung 4: Clock Tree des Mikrocontrollers [3, p. 93]	22
Abbildung 5: Schematische Darstellung One-Wire Bus	25
Abbildung 6: Timing Diagramm DHT11-Sensor [4, p. 5f]	25
Alle Abbildungen ohne Quellenangabe sind eigene Werke.	

Literaturverzeichnis

- [1 PlatformIO, „platformio.org“, PlatformIO, [Online]. Available:
] https://docs.platformio.org/en/stable/tutorials/ststm32/stm32cube_debugging_unit_testing.html#tutorial-stm32cube-debugging-unit-testing. [Zugriff am 08 August 2023].

- [2 ST Microelectronics, Medium-density performance line Arm®-based 32-bit MCU with 64
] or 128 KB Flash, USB, CAN, 7 timers, 2 ADCs, 9 com. interfaces DS5319 Rev 18, ST Microelectronics, 2022.

- [3 ST Microelectronics, Reference Manual for STM32F101xx, STM32F102xx,
] STM32F103xx, STM32F105xx and STM32F107xx advanced Arm®-based 32-bit MCUs RM008 Rev 21, ST Microelectronics, 2021.

- [4 Aosong Guangzhou Electronics Co., Ltd., Temperature and Humidity Module - DHT11
] Product Manual, Aosong.

Abkürzungsverzeichnis

Verwendete Abkürzungen

BPP	Betriebspraxisphase
CI/CD	Continuous Integration/Continuous Delivery (Deployment)
DUT	Device Under Test
GPIO	General Purpose In- and Output
HAL	Hardware Abstraction Layer
MCO	Microcontroller Clock Output
PIO	PlatformIO
RCC	Reset and Clock Control Register
RTOS	Real Time Operating System
SIE	System Industrie Electronic GmbH
VM	Virtuelle Maschine
VSC	Visual Studio Code

1 Praxisbericht der beiden Kompetenzbereiche

In dieser Betriebspraxisphase (BPP) wurde der Hauptaugenmerk auf die Bereiche Elektronik und Informatik gesetzt. Die vorangegangene BPP war damit belegt, dem Studenten wichtiges praxisbezogenes Wissen in projektartigen Arbeiten und Übungsaufgaben zu vermitteln. Zusätzlich wird der Student in dieser Phase auch das erste Mal in bei einem Kundenprojekt mitwirken.

1.1 Dispopapier

Zur Vollständigkeit ist hier noch einmal der Text des Dispopapiers.

Es sind drei Teilbereiche geplant in denen der Student seine Fertigkeiten verbessern soll.

1.: CI/CD Phase 2: Es wird in der Firma gerade die zweite Phase des Softentwicklungsprozesses eingeführt bzw. die Infrastruktur dafür aufgebaut und getestet. Im Laufe des Aufbaus wird der Student sich daher mit folgenden Themenbereichen beschäftigen: - Versionierung der einzelnen Branches auf der BitBucket-Plattform - Arbeiten mit Terraform - Zentralisierung, Zusammenführung, Filterung und Lifetime von Logfiles

2.: ETP-Testadapter: Das Projekt wurde bereits in den letzten Tagen der vorangegangenen BPP unterstützt. Hier wird die Unterstützung weitergeführt. Es gibt einen Testadapter bei dem die Firmware upgedatet werden und anschließend getestet werden muss.

3.: Vertiefung der Embedded Programmierkenntnisse: Mit dem STM32 Controller werden jede Menge Programmieraufgaben gestellt die der Student zu lösen hat. Hierbei soll die Ausführung der Programmierung sowohl mit als auch ohne Betriebssystem/Library erfolgen.

Erwartete Ergebnisse:

Zu 1.:

- *Verständnis für Struktur der CI/CD Pipeline*
- *Umgang mit den Plattformen Jenkins, Terraform, Docker, Azure, Jira, Confluence, Bitbucket*
- *Stabiles und sicheres Arbeiten mit der CI/CD Struktur*

Zu 2.:

- *Einsicht in das bereits bestehend Projekt*
- *Manueller Umgang mit Toolchains*
- *Herstellung verschiedener Schnittstellenkabel (CAN, USB, ADC, ICSP, ...)*
- *Neuerstellung der Dokumentation des Testadapters*

- *Umgang mit integrierter Hardware (uC ist auf Platine verbaut)*
- *Containerisierung der Toolchain*

Zu 3.: -

Bare Metal Programmierung

- *Clocks*
- *Externe und interne Interrupts*
- *I2C, OneWire Bus, PWM*

Mit RTOS und HAL

- *RTOS*
- *USB-Datenaustausch über das Protokoll JSON via PC*

Die einzelnen Teilbereiche werden in den nächsten Kapiteln genauer beschrieben.

2 Lernziele

Nachfolgend werden die Lernziele des Studenten in den einzelnen Disziplinen erläutert.

2.1 Elektronik

- Der Student kann Schaltpläne von elektronischen Baugruppen lesen und verstehen
- Hardwarespezifikationen können definiert als auch überprüft werden
- Es kann mit älteren Modellen von Arm-Controllern umgegangen werden
- Signale können gemessen, gelesen und interpretiert werden

2.2 Informatik

- Softwarespezifikationen können gelesen und interpretiert werden
- Mit Controllern der STM32F10x Serie können sowohl mit als auch ohne HAL einfache Aufgabenstellungen gelöst und realisiert werden
- Verständnis für die Continuous Integration/Continuous Delivery (CI/CD) Struktur
- Umgang und Teamarbeit mit Bitbucket
- Erstellung und Umgang mit manuellen, nicht-integrierten Entwicklungsumgebungen

3 Aufgabenstellung

3.1 CI/CD Phase 2

Um den Prozess der Softwareentwicklung und den damit verbundenen Aktionen weitestgehend zu automatisieren wird in der Firma System Industrie Electronic (SIE) eine CI/CD Struktur aufgebaut. Wie der Name schon sagt, dient diese der kontinuierlichen Integration und Auslieferung von Software an das Endgerät oder den/die Endgerätverwaltende*n Kund*innen. Im Rahmen des Aufbaus dieser Struktur wird der Student mit eingebunden, sodass dieser die neuen Abläufe und den Aufbau der Infrastruktur bereits in der zweiten Phase mitbekommt. In der ersten Phase wurde die Architektur für die Pipeline festgelegt, jedoch mit dem Vorbehalt für etwaige Änderungen. In dieser zweiten Phase werden alle Schnittstellen zwischen den einzelnen Abschnitten getestet, automatisiert, mögliche Probleme analysiert und eventuell direkt behoben.

Der Student soll hierbei mit den Tools bzw. Plattformen Docker, Terraform, Jenkins, Azure, Bitbucket, Sonatype Nexus und Jira vertraut werden.

Es wird gewünscht, einen Docker Container zu erstellen der die Entwicklung mit STM32 Controllern ermöglicht. Das so entstandene Image des Containers soll anschließend abgelegt und eingefroren werden.

3.2 ETP-Testadapter

Wie bereits im Betriebspraxisbericht 1 zu lesen ist, soll hier für ein Produkt eine Softwareänderung durchgeführt werden. Die Erstellung der Software muss Aufgrund ihres Alters und der bereits durchgeführten Validierung von bestimmten Tools durchgeführt werden, um eine minimale Änderung der Software zu ermöglichen. In der ersten Betriebspraxisphase wurde bereits versucht, den Quellcode des Produktes zu übersetzen, jedoch ohne Erfolg.

In dieser BPP soll der Student die Software abändern, übersetzen, auf das Gerät aufspielen und anschließend die durchgeführte Änderung testen.

Die Softwareänderung sowie alle weiteren gewonnenen Erkenntnisse sind hierbei exakt zu dokumentieren.

3.3 Vertiefung Embedded Programmierkenntnisse

Vom Student wird erwartet, dass er über die STM32 Controllerfamilie ein Grundwissen verfügt. Hierfür werden während der BPP2 einige Lehreinheiten von Michael Woog abgehalten. Zusätzlich dazu gibt es noch Übungseinheiten in denen die gelehrteten Inhalte angewendet werden. Abschließend gab es zu jedem Thema noch ein Review-Gespräch mit dem Betreuer,

in dem Probleme und deren Lösungen besprochen werden. Wie bereits im Dispopapier festgehalten, werden sowohl der direkte Adresszugriff über Memory Mapping als auch der Hardware Abstraction Layer (HAL) verwendet. Es sollen damit Interrupts, alternative Funktionen von GPIOs wie I2C, UART, PWM, o.Ä. eingestellt werden können. Das Betriebssystem „Real Time Operation System“ (RTOS) soll ebenfalls näher erläutert werden.

4 Erwartete Ergebnisse

4.1 CI/CD Phase 2

Wie in der Aufgabenstellung festgehalten wurde, wird ein funktionierender Docker Container als Ergebnis dieser Aufgabe erwartet. Durch die vorangegangene BPP ist der Grundsätzliche Aufbau eines Docker Containers klar. Allerdings ist der Umgang mit der Gesamtinfrastruktur zur Remoteentwicklung in einem Container noch nicht klar. Diese Kompetenz wird am Ende dieser BPP erwartet.

Zudem soll sich im Detail angesehen werden welche Arbeiten notwendig sind, um produktiv und effizient Software im Remotecontainer für STM32 Controller zu entwickeln.

4.2 ETP-Testadapter

Durch das Alter des Testadapter wird hier eine Einarbeitung in ältere Tools, nicht-integrierte Toolchains und eine Analyse der derzeit auf dem Controller aufgespielten Software erwartet. Wie bereits erwähnt sollen die so gewonnen Informationen über das Projekt genau dokumentiert werden.

Von Teamleiter Martin Tomasini wurde noch der Wunsch ausgesprochen, dass auch hier versucht werden soll einen Docker Container einzurichten. Mit diesem soll dann das Übersetzen der Quelldateien möglich sein.

4.3 Vertiefung Embedded Programmierkenntnisse

Es wird vom Studenten erwartet nach dieser BPP tiefere Einblicke in die Embedded Entwicklung mit Mikrocontrollern zu bekommen. Hierfür werden Lehreinheiten von Betreuer Michael Woog abgehalten. Dazu gibt es zu jeder Lehreinheit noch passende Übungen. Die Übungen werden dann vom Studenten erarbeitet und anschließend mit dem Betreuer besprochen. Etwas Problematisches im Umgang mit Mikrocontroller werden analysiert und verstanden. Der Student soll mögliche Lösungen im Problemfall erarbeiten.

5 Umsetzung

Zur Umsetzung der Aufgabenstellungen war der Student immer wieder an den beschriebenen Aufgaben am Arbeiten. Aufgrund von verschiedenen Dringlichkeiten, Wartezeiten auf Kollegen und Ähnlichem wurden die einzelnen Lernziele nicht historisch abgearbeitet. Details über die Erledigungszeitpunkte finden sich im Lerntagebuch wieder.

5.1 CI/CD Phase 2

Die Arbeiten des Studenten in der CI/CD Phase 2 gliedern sich in drei Unterpunkte:

- Verstehen der geplanten und teils bereits umgesetzten Infrastruktur und deren Teile
- Erstellung eines generischen Docker Containers für die STM32 Controller Familie
- Erstellung eines Docker Containers für die Entwicklung und Übersetzung der ETP-Testadapter Software

Zum besseren Verständnis der Infrastruktur ist in Abbildung 1 eine schematische Übersicht zu sehen. Zunächst werden die verwendeten Tools und deren Aufgaben, anschließend die einzelnen Abschnitte des automatisierten Prozesses erklärt.

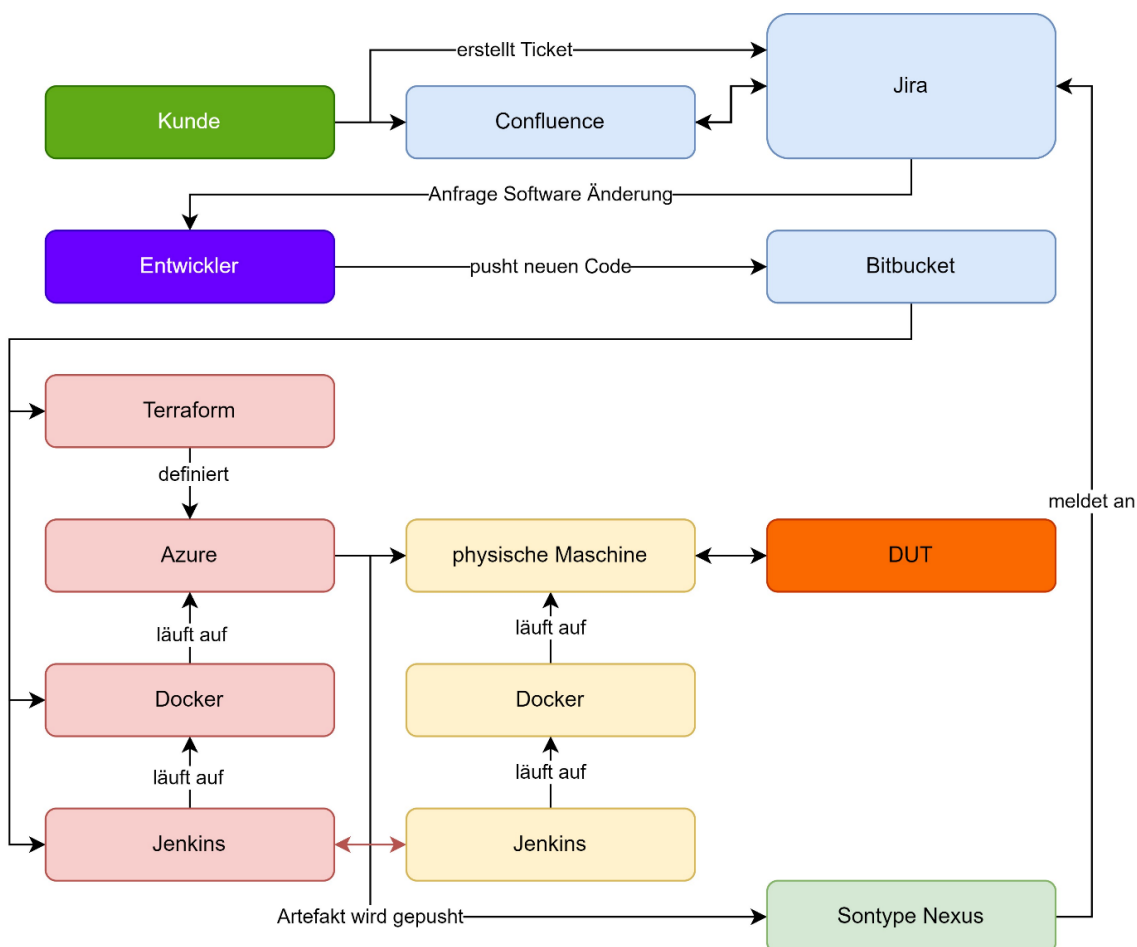


Abbildung 1: Aufbau der CI/CD Infrastruktur

5.1.1 Tools

Für das Verständnis der Infrastruktur ist es sehr wichtig die Tools und Plattformen die verwendet werden, zumindest im Ansatz zu verstehen. Hierfür werden die verwendeten Produkte kurz erklärt:

Jira ist ein Tool welches für Fehlertracking und Projektmanagement verwendet wird. Aus Anforderung oder Fehlerfällen werden Jira-Tickets erstellt die dann einer ausführenden Instanz zugewiesen werden.

Bitbucket ist eine Git basierende Versionierung-Plattform. Hier wird der Quellcode mit den dazugehörigen Informationen abgelegt und vom Team bearbeitet.

Confluence ist ein Dokumentierungswerkzeug mit Versionierung.

Terraform ist eine Infrastrukturbeschreibungssprache (infrastructure as code). Damit können Plattformunabhängig¹ die Ressourcen einer Infrastruktur und deren Konfiguration beschrieben werden.

Azure ist ein Webservice von Microsoft bei dem Ressourcen in der Cloud gemietet werden können.

Jenkins ist ein Tool mit welchem zuvor definierte Build- und Test-Jobs automatisch ausgeführt werden können. Es folgt dem klassischen Master/Agent Prinzip.

Sonatype Nexus ist ein Artefakt-Repository. Im Gegensatz zum Bitbucket Quellcode-Repository ist dieses hier auf Artefakt Versionierung und Deployment bzw. Delivery² spezialisiert.

Docker ist ein Werkzeug, dass zur Abstrahierung der Systemumgebung dient. Ähnlich wie auf einer VM kann dort Software ausgeführt werden. Docker ist allerdings wesentlich schlanker als die VM-Lösung, da nicht auf Hardwareebene, sondern auf Applikationsebene abstrahiert wird.

Der Vorteil einer Docker-Umgebung ist seine Konsistenz: egal auf welchem Hostsystem die Engine läuft, für die in Docker laufende Software sieht die Umgebung immer exakt gleich aus.

5.1.2 CI/CD Standardprozedur

Der Standardprozess soll im Allgemeinen wie folgt ablaufen:

¹ Terraform ist bedingt Plattformunabhängig. Werden Plattformspezifische Services von bspw. Azure verwendet so sind diese nicht ohne weitere portierbar.

² Deployment beschreibt den automatischen Auslieferungsprozess bis zum Endgerät. Delivery beschreibt den automatischen Auslieferungsprozess bis zum Kunde der Software. Diese muss hier manuell auf das betreffende Gerät gespielt werden.

Der/die Kund*innen oder durch den/die Kund*innen in Kenntnis gesetzte Systemarchitekt*innen (nachfolgend Initiator*innen genannt) erarbeiten neue Anforderungen für eine bestehende oder neue Software. Diese Anforderungen werden von den Initiator*innen in ein Jira-Ticket umgewandelt und dann einem entwickelnden Team mit den entsprechenden Kompetenzen zugeordnet. Das entwickelnde Team adaptiert oder kreiert nun Software die den Anforderungen entsprechen. Die so entstandene Software wird nun auf das Bitbucket-Repository gepusht. Es wird nun automatisiert erkannt, dass neuer Quellcode zur Verfügung steht. In den Quellcode Dateien befinden sich außer dem Quellcode des Device Under Test (DUT) auch die Anweisungen und Beschreibungen für die anderen beteiligten Instanzen wie Terraform oder Jenkins.

Es wird nun durch die Quellcodeänderungserkennung die Prozedur angestoßen und der Prozess gestartet. Zuerst wird auf dem Azure Host eine Virtuelle Maschine (VM) mit allen benötigten Schnittstellen und Systemressourcen angelegt. Die Beschreibung der Ressourcen und Schnittstellen befinden sich im Terraform Skript.

Diese so definierte VM startet nun den ebenfalls im Quellcode abgelegten Docker Container. Je nach Änderung des Docker Quellcodes wird entweder der Container direkt gestartet oder zuerst noch ein Image für den Container erstellt und dieses anschließend gestartet. Auf diesem Docker Container läuft dann eine Jenkins-Build-Applikation.³ Ist der Build erfolgreich abgeschlossen wird das Artefakt mit dem Hinweis der ausstehenden Testung im Sonatype Nexus Repository abgelegt.

Nun folgt die automatisierte Testung. Hierfür wird auf einer physikalischen Maschine ebenfalls ein Docker Container mit Jenkins gestartet. Der dort laufende Test-Jenkins wird aus der in Azure laufenden Instanz gesteuert und führt so am DUT seine vorgegebenen Tests durch. Wurden alle Tests abgeschlossen, so wird der Testreport mit dem Artefakt abgelegt. Nun können die zuständigen Initiator*innen entscheiden, ob der Test als erfolgreich oder nicht erfolgreich abgeschlossen wurde und die Software weiter verwalten.

Die SIE arbeitet im Bereich der eingebetteten Systeme und ist zudem ein Dienstleister welcher Geräte für Kunden entwickelt. Durch die kleinen Stückzahlen und der hohe Aufwand des automatisierten Deployments wird zunächst die Delivery-Struktur aufgebaut und eingesetzt. Das Deployment ist vorgesehen, aber wird noch nicht aktiv verfolgt. Hier wird/werden Initiator*innen lediglich darüber informiert, dass ein Update für ihr Produkt verfügbar ist.

³ Die Applikation besteht aus mehreren Jenkins Agents und einem Master.

5.2 Erstellung eines generischen Docker Containers für STM32 Controller

Zunächst war noch nicht klar, ob Visual Studio Code (VSC) oder die Herstellerspezifische STM32 CubeIDE verwendet wird. Die CubeIDE ist eine auf Eclipse basierende IDE die von ST angepasst wurde. Mit ihr ist es sehr einfach und intuitiv Software zu erstellen und zu debuggen. Außerdem ist das Cube Framework sehr gut in die IDE eingebunden. So können sehr einfach Clock Einstellungen oder Schnittstellentreiber wie I2C, USB oder beispielsweise UART initialisiert und verwendet werden. Die IDE hilft auch bei möglicher Fehlkonfiguration der einzelnen Funktionen. Wird etwa ein Pin bereits von einem Schnittstellentreiber verwendet so kann dieser nicht mehr als IO-Pin verwendet werden. In der GUI von Cube sind dann die entsprechenden nicht mehr verfügbaren Funktionen gekennzeichnet.

VSC ist ein Codeeditor mit jeder Menge Plugins. Durch die Herstellerunabhängigkeit können sehr viele Chip-Typen damit programmiert und debuggt werden. Für die Programmierung und das Debugging sind zwei oder mehr Ansätze möglich. Es könnte etwa über das integrierte Terminal jede Aktion manuell ausgeführt werden. Aufgrund der Benutzerfreundlichkeit wurden allerdings passende Plugins gesucht, um Arbeitsschritte zu automatisieren.

5.2.1 Vorgehen

Zunächst wurde versucht, die STM32 CubeIDE auf der Docker Umgebung einzurichten. Hierbei kristallisierte sich bereits das erste Problem heraus: die CubeIDE ist eine GUI basierte IDE. Daher ist es nötig, dass im Docker-Container ein Server aktiv ist, welcher die anfallenden Bilddaten über einen Kanal veräußert.

Es wurde versucht, mit einem xVNC-Server die Bilddaten an das Host System weiterzugeben, allerdings hat dies nicht funktioniert. Da die alternative mit VSC bereits bekannt war, wurde in diesen Ansatz nicht viel Zeit investiert und schnell in Richtung VSC gearbeitet.

Für VSC gibt es sehr viele Plugins für die Mikrocontrollerentwicklung. Wenn allerdings das STM Cube Framework verwendet werden soll, so schrumpft die Auswahl auf unter anderem folgende Möglichkeiten:

- VSC mit PlatformIO (PIO) als IDE
- VSC als Highlighting Texteditor, Artefakt-Erstellung mit Maketool (Make oder cmake) und Gnu Debugger (GDB) Server mit Command Line Interface (CLI) als Debugging Tool

In beiden Fällen kann zur Erstellung des Projekts der STM eigene Codegenerator CubeMX verwendet werden. CubeMX ist das GUI-Tool, welches in die CudeIDE integriert ist. Mit MX ist es sehr angenehm Hardwarekonfigurationen vorzunehmen. Die eingestellten Konfigurationen können dann als Code exportiert werden.

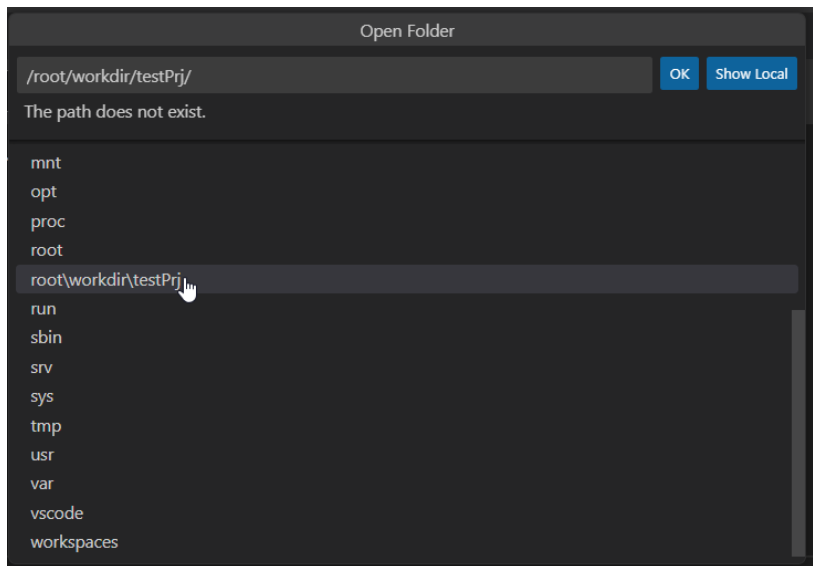
Zunächst wurde PIO verwendet, um ein Programm anzulegen und zu debuggen. Um die einwandfreie Funktionalität zu gewährleisten und den Umgang mit PIO kennenzulernen wurde dies zunächst lokal auf dem PC ausprobiert. PIO stellt eine Schritt-für-Schritt Anleitung für ST-Mikrocontroller auf ihrer Seite zur Verfügung. [1] Dieser Anleitung gefolgt konnten sofort alle Anforderungen eingehalten werden. Nun sollte die gleiche Umgebung mit Docker realisiert werden.

Zur Umsetzung wurde ein DevContainer für VSC verwendet. Ein DevContainer ist ein Container, in dem eine VSC-Server Instanz läuft. In der DevContainer beschreibenden Datei werden alle Plugins und ihre Konfigurationen inklusive den benötigten Bibliotheken festgehalten. Da das DevContainer Image einmal mit allen zugehörigen Plugins und Bibliotheken gebaut wird, ist der Stand zum Zeitpunkt des Imaging aller Bibliotheken und Abhängigkeiten eingefroren. Dies ist äußerst hilfreich in der Softwareentwicklung, da oftmals Programmteile auf anderen aufbauen. Wenn nun der unterliegende Programmteil z.B. vom Hersteller der Bibliothek geändert wurde, kann es dazu kommen, dass einst funktionierende Software nicht mehr übersetzt werden kann oder nicht mehr wie einst geplant und getestet funktioniert. Um dieses Problem zu verhindern, wird der DevContainer verwendet. Auf dem lokalen PC muss dann lediglich eine VSC-Client Instanz und der DevContainer in der Docker Engine laufen⁴.

Das Programmieren, Debuggen und Überspielen funktionierte im DevContainer ohne weitere Probleme. Um gebaute Artefakte direkt auf dem Lokalen System zu haben und ablegen zu können, wurde ein Ordner in den Container gemountet. Auch das hat funktioniert. Probleme gibt es allerdings beim Erstellen neuer Projekte.

Wenn das Host System auf dem der DevContainer läuft, windowsbasierend ist, wird bei der Pfaderstellung der falsche Slash-Typ verwendet. Anstelle von , / ' wie bei unixoiden Systemen üblich, wird der Backslash von Windows verwendet. Dargestellt in Abbildung 2. Das so erstellte Projekt hat einen Pfad in dem Backslashes vorkommen. Dieses kann nun nicht mit PIO geöffnet werden. Ein Workaround hierfür ist, das Projekt mit mittels CLI im Container in einen Backslash-freien Pfad zu kopieren und anschließend zu öffnen.

⁴ Die Docker Engine kann auch auf einer nicht-lokalen Ressource laufen.



Um das Problem zu beheben wurde Kontakt mit der PIO-Community aufgenommen. Hier wurde der Student auf die GitHub Seite von PIO verwiesen, um das Problem zu melden. Das Ticket ist zum Zeitpunkt der Erstellung dieses Berichts noch nicht bearbeitet, geschweige denn gelöst worden.

Abbildung 2: PlatformIO verwendet den falschen Slash

Dieses und noch ein paar andere Probleme beim Debuggen führten zu der Entscheidung, für die interne Entwicklung nicht PIO in Kundenprojekten zu verwenden.

Als nächstes wurde der Weg der manuell eingerichteten IDE verfolgt. Der Student hat versucht, Code mit CubeMX zu erzeugen und diesen dann in der DevContainer Umgebung zu übersetzen und überspielen. Das Einbinden der Bibliotheken des Cube Frameworks bereitete hierbei Probleme. Es wurde noch ein wenig Zeit darin investiert, allerdings wurde dann der Fokus auf andere Punkte gelegt und die Zeit in der BPP2 ging zu Ende.

5.3 Erstellung eines Docker Containers zum ändern der Testadapter Software

Um die Softwareänderung am Testadapter durchzuführen, musste eine Entwicklungsumgebung erstellt werden, mit dem der Code aus dem Jahr 2007 übersetzt werden kann. Es ist allerdings so, dass für diesen Code einige Bibliotheksteile zusammenkopiert und adaptiert wurden. Daher wurden diese nicht mehr gewartet. Im Code findet sich mehrmals die Attribute „PROGMEM“ in den Sektionen des USB-Treibers. Neuere AVR-GCC Compiler unterstützen diese Attribute, allerdings nur wenn die Variable als Konstant qualifiziert wird. So wurde eine Umgebung benötigt, in der ein AVR-GCC Compiler läuft, der ohne diese Qualifizierung auskommt.

In der Dokumentation des Testadapters war der verwendete Compiler erwähnt, es handelte sich um den Win-AVR-GCC-2007. Hier stellt sich allerdings bereits das nächste Problem in den Weg: üblicherweise sind die Docker-Container unixoide System und keine DOS-basierenden. Win-AVR-GCC ist allerdings ein Compiler der ausschließlich auf Windows Systemen läuft. Es wurde nun ein AVR-GCC Compiler gesucht, der zeitlich aus der Ära stammt und bei dem die PROGMEM Attribute kein Problem darstellt.

Das Docker-Image wurde dann über mehrere Iterationen versucht zu erstellen. Es gab mit neueren Ubuntu Versionen im Docker-Image das Problem, dass hier Abhängigkeiten nicht gepasst haben. Schließlich wurde das Docker-Base-Image von Ubuntu 08.04 Navit verwendet. Hier konnte der AVR-GCC Compiler installiert werden. Die Quelldateien konnten so ohne weiteres übersetzt werden, allerdings waren die erzeugten Binärdateien von unterschiedlicher Größe. Die so erzeugte Binärdatei wurde daher nicht im Kundenprojekt eingesetzt. Zu Übungszwecken wurde allerdings noch die automatisierte Artefakt-Erzeugung und Speicherung eingerichtet.

Zunächst wurde der Docker-Container über ein Shell-Skript gebaut und gestartet. Im Docker-Container wurden dann automatisch alle Quelldateien in den Container kopiert und nach erfolgreichem Übersetzen die Artefakte wieder zurück auf das Hostsystem kopiert. Um nicht jedes Mal alles in den Container und wieder herauskopieren zu müssen, wurde versucht ein Volume in den Container zu mounten. Nach mehreren Versuchen wurde dieses Vorhaben eingestellt.

Es wird vermutet, dass die Inode Größen des Ubuntu 08.04 und Windows 10 Betriebssystems nicht zusammenpassen. Diese Information wurde allerdings nicht ausreichend verifiziert.

5.4 ETP-Testadapter

Aufgrund eines Re-Designs einer Platine eines Kunde musste die Software eines ETP-Testadapters angepasst werden. Es handelt sich um den gleichen Änderungsauftrag der bereits vorangegangen BPP.

Zunächst musste geklärt werden, wie mit den Fuse und Lock-Bits beim Controller umgegangen werden muss. Es ist ein Controller der Firma Atmel. In der Dokumentation ist ersichtlich wie diese Bits geschrieben und gelesen werden können. Die Prozedur, mit der auf den entsprechenden Speicherbereich zugegriffen werden kann, ist nicht trivial, so dass von einem manuellen beschreiben abgesehen wurde. Wie sich herausstellte bietet das Tool AVR-Dude genau diese Funktionalität. Es können Lock- und Fusebits gelesen und geschrieben werden.

Das exakte Vorgehen beim flashen und etwaige Probleme wurden von Michael Woog analysiert. Dabei kam zum Vorschein, dass das EVAL-Board für den Controller programmiert und auch der Debug-Modus aktiviert werden kann. Wird allerdings der Debug-Modus beim Controller im Testadapter aktiviert so kann dieser nicht wieder deaktiviert werden. Der Grund dafür ist, dass an den entsprechenden Pins zur Umschaltung des Modus bereits ein Peripherie-Controller hängt der nicht abgeklemmt werden kann. So kam es dazu, dass bei den Adaptern mit aktiviertem Debug-Modus, die Controller ersetzt werden mussten.

Als nächstes wurde vom Studenten analysiert werden mit welcher Softwareänderung die gewünschte Funktion erreicht werden konnte. Zunächst wurde die Software genauestens gelesen, um ihre Funktionsweise zu verstehen. Anschließend wurden einige Tests mit dem

Testadapter am Kundengerät durchgeführt. Es kamen dabei einige Fehler ans Tageslicht die alle behoben werden konnten.⁵

Mit diesem Wissen konnte nun die Softwareänderung an die Geräte der Produktion ausgerollt werden. Es wurden in diesem Zug etwa 150 Testadapter mit der neuen Software bespielt und kalibriert.

5.5 Vertiefung Embedded Programmierkenntnisse

Für die Vertiefung der Programmierkenntnisse wurden von Michael Woog einige Aufgaben zur Verfügung gestellt. Hier eine Übersicht der Übungsaufgaben:

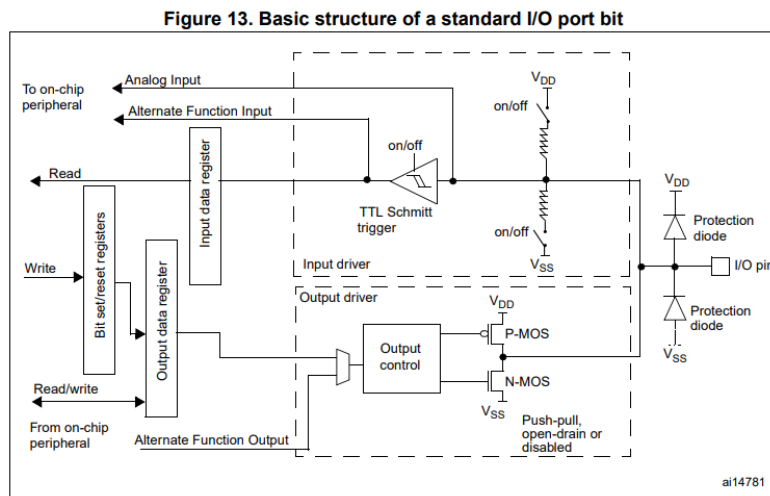
- Konfiguration der einzelnen Clock Prescaler und Multiplexer
- Ansteuerung des Microcontroller Clock Output (MCO)
- Konfiguration des System Tick Timer
- Implementierung des Protokolls für den LCD 1602 Punkt-Matrix-Bildschirm
- Implementierung des Protokolls für den DHT11 Feuchtigkeitssensor

Alle Aufgaben sollten ohne Einsatz der HAL als Bare-Metal-Implementierung mit direktem beschreiben der Register durchgeführt werden. Der verwendete Controller war ein STM32F103C8T6 BluePill Evaluations-Board. In den folgenden Seite wird auf Probleme und neue Erkenntnisse bei der Umsetzung der Übungsaufgaben eingegangen. Zudem wird die Funktionsweise der Umsetzung erklärt.

5.5.1 Ansteuerung des Mikrocontroller Clock Output

⁵ Um die Interessen der Firma SIE und deren Kunden zu wahren können Details dieser Arbeit nicht veröffentlicht werden. Da möglicherweise erkennbar ist, um welches Produkt oder welchen Kunden es sich handelt werden Technische Lösungen und Tätigkeiten nur oberflächlich beschrieben und teilweise ganz weggelassen. Im Rahmen der Beurteilung ist eine Einsicht nach Absprache mit der Firma SIE möglich.

Zunächst folgt der Clock-Tree des Controllers in Abbildung 4. In dieser Übung sollte der MCO eingestellt und aktiviert werden. Der MCO ist dabei eine sogenannte „Alternate Function“. Wie



in der Abbildung 3 ersichtlich ist, muss bei diesem Controller der Output Treiber auf die Alternate Function gemappt werden. Daher muss in der Liste der Alternate Function nachgesehen werden, wohin dieser MCO physisch gemappt wird. Wie im Datenblatt des Controllers ersichtlich ist, wird der MCO auf Pin A8 gemappt. [2, p. 31]

Abbildung 3: Aufbau der Pin Ein- und Ausgangsstufe [3, p. 160]

Zur Umsetzung muss der MCO aktiviert und eine Clock-Quelle angegeben werden. Außerdem muss der Pin A8 im GPIO Control Register als Alternate Function konfiguriert werden. Nun wird der Pin A8 vom Controller mit der eingestellten Frequenz geschaltet.

5.5.2 Konfiguration der einzelnen Clock Prescaler und Multiplexer

In dieser Übung war das Ziel, die Peripherie Bus Clocks einstellen zu können. In Abbildung 4 sind diese als APB1, APB2 und HCLK bzw. AHB zu sehen. Es soll auch eine Überprüfung auf Übertaktung stattfinden.

Der Student hat hier eine Funktion geschrieben, der ein Strukturzeiger übergeben wird. In dieser Struktur befinden sich die ganzen Pfadinformationen der Taktgeber. So können die beiden Multiplexer PLL Source und System Clock eingestellt werden. Die drei Prescaler Informationen und die Frequenz des Externen Oszillators müssen ebenfalls angegeben werden. Wird nun die Funktion mit der Struktur aufgerufen so werden zunächst die Parameter über-

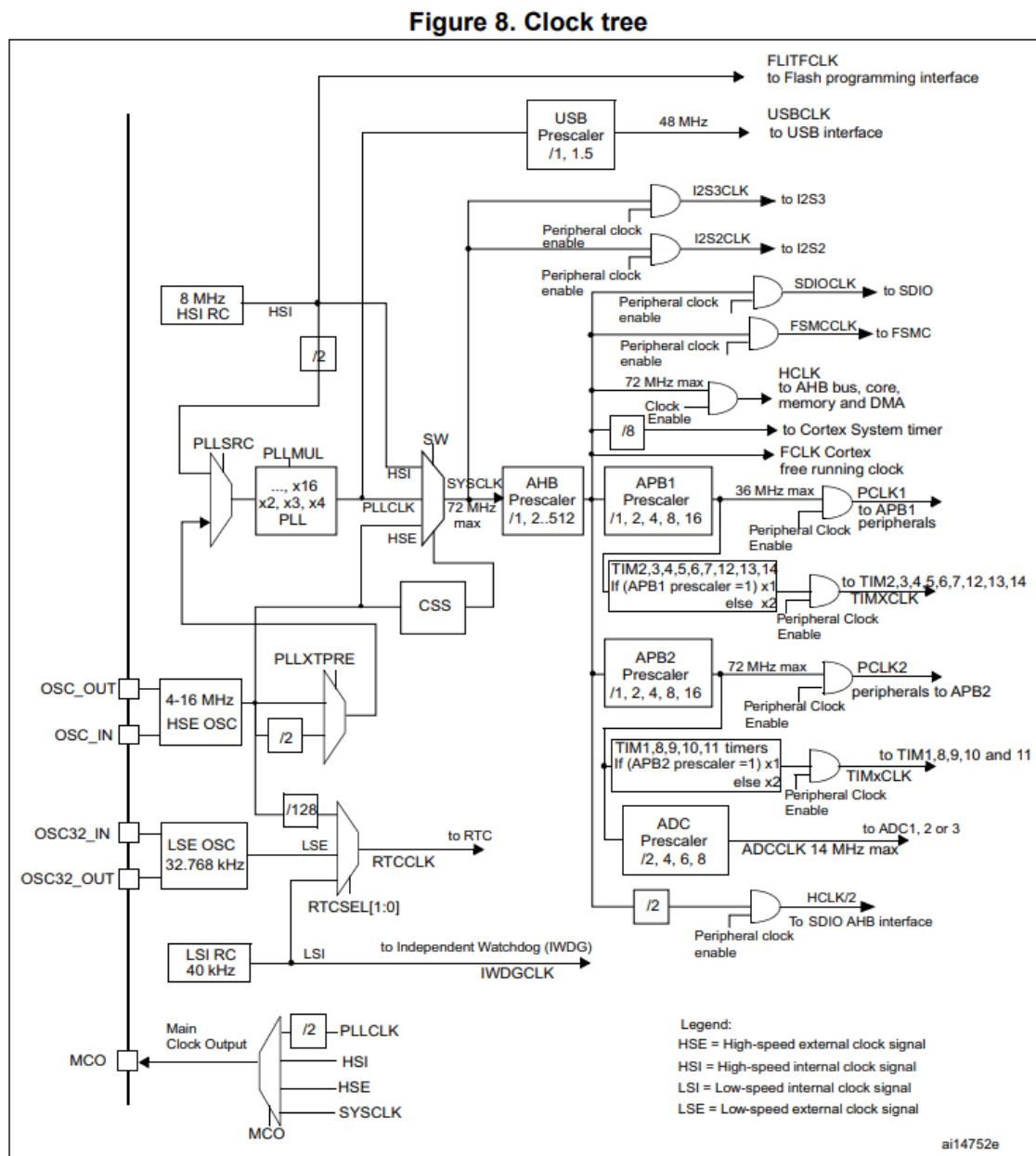


Abbildung 4: Clock Tree des Mikrocontrollers [3, p. 93]

prüft. Wenn diese in Ordnung sind und keine Übertaktung stattfindet so werden die Clocks eingestellt. Sollte dies nicht der Fall sein wird eine Fehlermeldung zurückgegeben.

Es wird auch das Clock Security System eingeschaltet, wenn von Nöten. Das ist der Fall, wenn der externe Oszillator direkt oder über die PLL als Clock-Quelle genutzt wird. Nach

erfolgreichem Einstellen aller Clocks wird die eingestellte Sysclock Frequenz in die Struktur geschrieben und eine Gut-Meldung zurückgegeben.

Im Zuge der Übung wurde auch noch versucht, was passiert, wenn die zeitlichen Spezifikationen nicht eingehalten werden. Im Reference Manual steht, dass der Pin für den MCO mit maximal 50MHz betrieben werden darf. Es wurde nun eine höhere Frequenz eingestellt. Am Oszilloskop war dann ersichtlich, dass sich das Clock-Signal immer mehr verzerrte. Bei 72MHz Ausgangsfrequenz wurde das Clock-Signal zu einer Sinusartigen Schwingung mit einer Amplitude von etwa 0,7V und einem DC-Offset von etwa 1,8V anstelle des eigentlichen Signals mit 0 - 3,3V.

5.5.3 Konfiguration des System Tick Timer

Wie in Abbildung 4 gesehen werden kann, wird der SysTick Timer⁶ entweder mit der HCLK-Frequenz oder der HCLK-Frequenz / 8 betrieben werden. Für die Einbindung des SysTick Timer Interrupts wurde eine vorgefertigte Funktion von Michael Woog verwendet. Der Student musste anschließend noch eine Funktion schreiben welche das Nachladeregister des SysTick Counters beschreibt und gegebenenfalls das Register setzt, um die HCLK-Frequenz durch 8 zu teilen. Im Zuge der Übung sollte ein System Tick von 1 Mikrosekunde eingestellt werden.

Schon vor dem Starten des Programms war für den Studenten klar, dass dies zu Konflikten führen kann. Wenn die System Clock (nicht SysTick) mit standardmäßig 8MHz betrieben wird und der SysTick Interrupt mit 1MHz Takten soll, so sind nur 7 Takte zwischen ISR und normaler Programmausführung verfügbar. Mit einem System Clock von 8MHz konnte der Controller nicht mehr betrieben werden. Der System Clock wurde daher mit 72MHz betrieben. Selbst bei dieser Frequenz sind nur maximal 71 Takte zwischen ISR und normaler Programmausführung. Wenn man bedenkt, dass bei jedem Interrupt noch der momentane Programmstatus gesichert und nach der ISR wiederhergestellt werden muss, so sind nicht mehr viele Takte für die Programmausführung übrig.

Mithilfe dieses SysTicks wurde eine Funktion erstellt die den Programmablauf für eine parametrisierbare Anzahl an Mikrosekunden unterbricht. Die Funktion wird in den nachfolgenden Beispielen verwendet.

5.5.4 Erstellung eines Moduls für den LCD 1602 Punkt-Matrix-Bildschirm

Der LCD 1602 wird über einen parallelen Bus angesteuert. Mit einer Flanke am Enable Pin werden die Daten übernommen. Der LCD kann mit einem 4 oder 8 Bit breitem Bus betrieben werden. In der Startprozedur des Bildschirms wird festgelegt, welche Busbreite verwendet wird. Der Student hat hier ein Programm geschrieben, welches den Bus mit den erforderlichen

⁶ Die Leitung Cortex System Timer führt zum SysTick Counter.

Daten bedient. Die Busbreite und der verwendete Port können mithilfe einer Definition eingestellt werden. Durch die bedingte Kompilierung werden nur die Teile übersetzt die für die Applikation notwendig sind.

Es wurden die Funktionen zum Setzen des Cursors an eine Position, Löschen des Bildschirms und Rückkehren des Cursors an die Home Position und Schreiben von einzelnen Standardzeichen, benutzerdefinierten Zeichen oder Strings implementiert. Außerdem kann auch die Cursoranzeige de- und aktiviert werden. Selbes gilt für das Blinken des Cursors.

Um benutzerdefinierte Zeichen zu schreiben, müssen davor die Matrizen für die Bildpunkte übertragen werden. Diese werden dann in den Character Generator RAM (CGRAM) geschrieben. Von dort aus können die Zeichen dann verwendet werden.

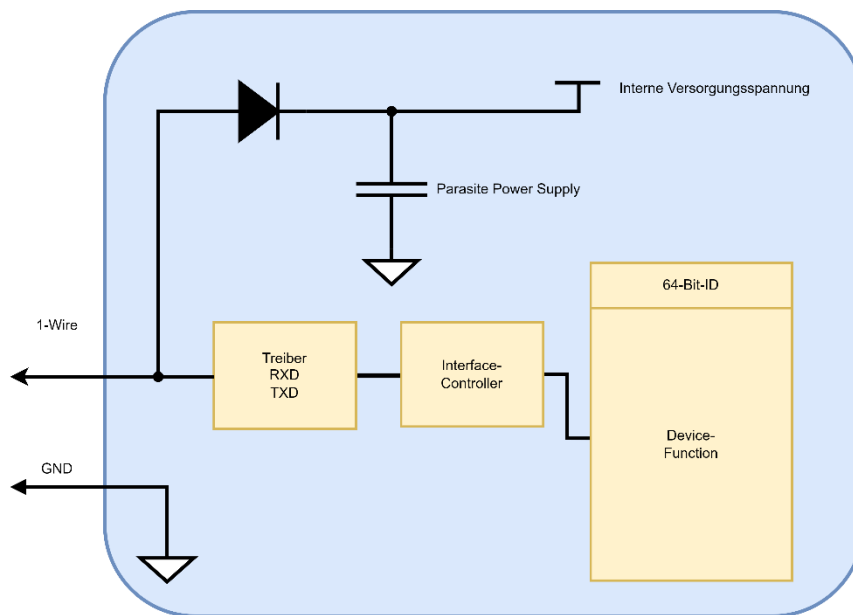
Ein Problem bei der Umsetzung war noch, dass die verwendeten Kabel und Breadboards äußerst schwache Verbindungen hatten. Dadurch wurde die Datenkommunikation anfangs gestört. Nach dem bessere Kabel verwendet wurden, war das Problem gelöst.

Ein weiteres Problem war, dass bei der Verwendung des 4 Bit Busses teilweise keine Sinnvollen Daten mehr übertragen werden konnten. Ein Neustart von LCD und Controller behob den Fehler stets. Bei der Analyse des Problems wurde festgestellt, dass es immer dann auftritt, wenn der Reset Button oft und schnell hintereinander gedrückt wird. Daher wird davon ausgegangen, dass immer dann ein Problem entstand, wenn der Controller neu gestartet hat und bereits in Nibble auf dem Bus übertragen wurde. Das Display war dann immer um ein halbes Byte daneben. Nur ein Neustart des Bildschirms half den Fehler loszuwerden.

5.5.5 Erstellung eines Protokolls für den DHT11 Feuchtigkeitssensor

Der Student hat mit dieser Aufgabe begonnen, doch hat sie aus Zeitgründen nicht fertiggestellt. Die Aufgabe und der momentane Status wird allerdings erklärt.

Der DHT11 ist ein Feuchtigkeitssensor der über ein One-Wire-Bus Protokoll kommuniziert. Der Bus kommt dabei mit nur einer Daten- und einer Ground-Leitung aus. Wenn Stromsparende Geräte wie der DHT11 verwendet werden, so kann die VCC-Leitung sogar wegfallen, da über die Datenleitung die auf VCC steht, genug Leistung in einem Kondensator gespeichert werden kann. Die so zur Verfügung stehende Leistung wird dann während der Datenübertragung verwendet. Nach erfolgter Datenübertragung kann der Kondensator wieder geladen werden. Das Schema ist in Abbildung 5 ersichtlich.



Das Protokoll des One-Wire-Buses ist durch zeitliche Impulse codiert. Das Timing-Chart für die Datenübertragung und die Bit-Codierung von 0 und 1 ist in Abbildung 6 zu sehen.

Die Aufgabe konnte aus Zeitgründen nicht abgeschlossen werden. Je nachdem wie die Wartefunktion für Mikrosekunden im Programm aufgerufen wird, werden andere Messergebnisse bei gleichem Signal aufgezeichnet. Der Grund dafür wird in der ISR vermutet. Wie bereits zuvor beschrieben wird die System Tick ISR im Mikrosekundentakt aufgerufen. Es konnte aber noch nicht zweifelsfrei festgestellt werden, warum das derzeitige Programm nicht funktioniert.

Abbildung 5: Schematische Darstellung One-Wire Bus

angefordert wird, werden andere Messergebnisse bei gleichem Signal aufgezeichnet. Der Grund dafür wird in der ISR vermutet. Wie bereits zuvor beschrieben wird die System Tick ISR im Mikrosekundentakt aufgerufen. Es konnte aber noch nicht zweifelsfrei festgestellt werden, warum das derzeitige Programm nicht funktioniert.

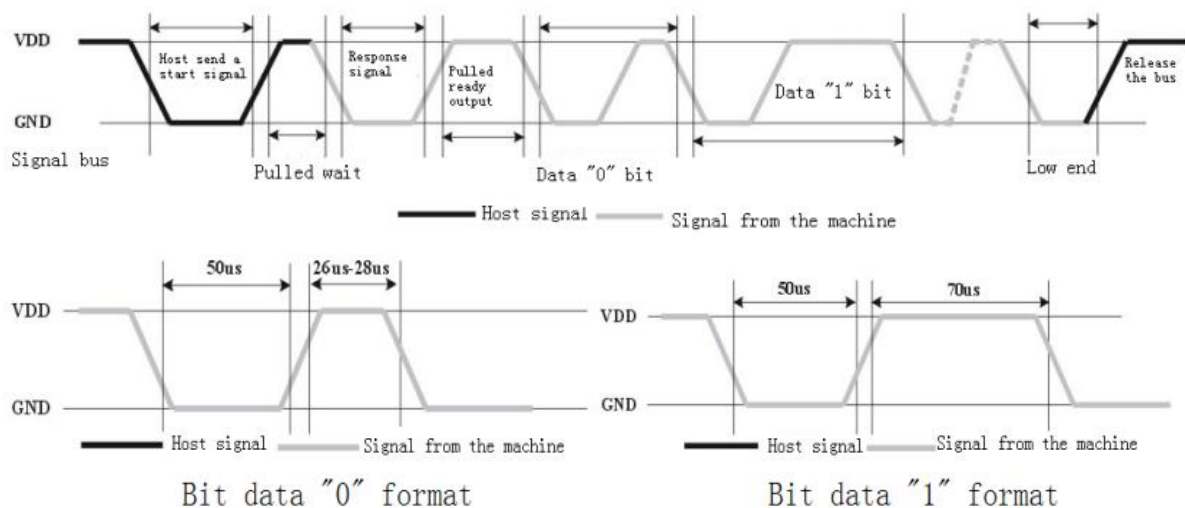


Abbildung 6: Timing Diagramm DHT11-Sensor [4, p. 5f]

6 Gegenüberstellung der erwarteten Ergebnisse und des Erreichten - Zusammenhang Studieninhalte und Praxisinhalte

Das Ziel der Einarbeitung in die CI/CD Infrastruktur wurde erreicht. Der Student kann einerseits die Infrastruktur beschreiben und andererseits hat er sich einiges an Wissen über Docker Containerisierung aneignen können. Er konnte sich einen Einblick in Azure und Terraform verschaffen. Da dieses Projekt allerdings noch nicht abgeschlossen ist, ist es nicht möglich die Anwendung damit zu praktizieren. Die Jenkins-Software wurde in dieser BPP noch nicht verwendet.

Der Student hat im Zuge dieses Projekts den Softwareingenieur Benjamin Hartmann bei einigen Problemen unterstützt. Eine Einarbeitung in Sonatype Nexus war Aufgrund des Projektstatus noch nicht möglich. Das Artefakt-Repository wird gerade erst eingerichtet und ist daher noch nicht verfügbar.

Die Unterstützung im Punkt ETP-Testadapter wurde voll erreicht. Der Student hat die Software für die Produktion verändert, getestet und die Dokumentation dafür erstellt. Zudem wurde das Bitbucket Repository für dieses Projekt vom Studenten erstellt und gepflegt. Nun können mitarbeitende jederzeit vollumfänglich nachvollziehen warum welche Schritte gesetzt und durchgeführt wurden.

Das Ziel der Vertiefung der Programmierkenntnisse wurde teilweise erreicht. Aus Zeitmangel konnten leider ein paar Punkte nicht erledigt werden. Der Student konnte allerdings sehr viel über die Bare-Metal-Programmierung an STM-Controllern von Michael Woog lernen. Er ist nun imstande ohne HAL weitreichende Konfigurationen beim STM32F103C8T6 Controller vorzunehmen.

Eine erweiterte Anwendung der HAL konnte leider nicht mehr vermittelt werden. Es wurde auch kein Datenaustausch über USB realisiert. Beide Punkte konnten aus Zeitgründen nicht erledigt werden.

7 Resümee der Betriebspraxisphase: Ausgeführte Tätigkeiten, Lernzielerreichung, persönliche Erfahrungen – Reflexion und Selbsteinschätzung

In dieser BPP lag der Fokus bei Elektronik und Informatik. Aufgrund der Arbeit an den einzelnen Bauteilen hat sich mein Wissen in den Bereichen vertieft.

Bei der Unterstützung in der zweiten Phase Umsetzung der CI/CD-Struktur konnte ich mich mit den einzelnen Tools auseinandersetzen. Einige der Probleme im Zusammenhang mit Docker-Containern wurden mir klar, andere konnte ich beheben. In Zusammenarbeit mit Benjamin Hartmann wurden auch Netzwerkprobleme und Sicherheitsaspekte beleuchtet. Etwa der Zugriff auf Ressourcen mittels SSH.

Durch die Arbeit an der ETP-Testadapter-Software hat sich mein Wissen für ältere Mikrocontroller stark verbessert. Ich kann nun nicht-integrierte Toolchains analysieren, verwenden und adaptieren. Mein Erfahrungsschatz ist hierbei sehr stark angewachsen. Außerdem mussten bei dem Projekt einige Analysen durchgeführt werden. Dies hat mir sowohl aus informatischer als auch elektronischer Sicht sehr viel gelehrt.

Im Bereich der Mikrocontrollerprogrammierung konnte ich mein Wissen festigen und erweitern. Durch Michael Woogs Übungen für den STM32 Controller habe ich wichtige Einblicke in den Ablauf und die Programmierung bekommen. In den Übungen wurden Hardwarenahe Probleme behandelt und Lösungen dafür erarbeitet. Durch die Aufgabenstellung gab es auch in manchen Bereichen Stolperstellen für die Lösungen erarbeitet werden mussten. Durch diese Erarbeitung hat sich mein Wissen stark verbessert.

In dieser BPP konnte ich viel Kenntnisse in den zuvor vereinbarten Teilbereichen gewinnen. Auch wenn nicht alle Punkte des Dispopapiers erledigt werden konnten, so konnte ich dennoch weitreichende Einblicke in viele Teilbereiche gewinnen.

Position im Betriebspraxisbericht	%	Gewichtung	% gew.
Bericht an die Dualbetreuer der FH Vorarlberg	90	0,2	10
Praxisbericht der einzelnen Kernbereiche	100	0,35	17,5
Resümee über die Betriebspraxisphase 1: Ausgeführte Tätigkeiten, Lernzielerreichung, persönliche Erfahrungen – Reflexion und Selbsteinschätzung	100	0,25	12,5
Zusammenfassung Lerntagebuch, Zusammenstellung Abwesenheiten und Sorgfalt in der Ausarbeitung	100	0,20	10
Summe	392	1,00	50
Gesamtnote	1		

8 Zusammenfassung des Lerntagebuchs

Um die Interessen der Firma SIE und deren Kunden zu wahren können manche Details dieser Arbeit nicht veröffentlicht werden. Da möglicherweise erkennbar ist, um welches Produkt oder welchen Kunden es sich handelt. Technische Lösungen und Tätigkeiten werden partial nur oberflächlich beschrieben oder ganz weggelassen. Im Rahmen der Beurteilung ist eine Einsicht nach Absprache mit der Firma SIE jederzeit möglich.

Die entsprechenden Passagen sind in diesem Kapitel mit der Fußnote „Interessenswahrung“ versehen.

8.1 Zusammenfassung KW27

Ausgeführte Tätigkeiten:

In der ersten Arbeitswoche wurde zunächst der Arbeitsplatz eingerichtet. Es wurde in dieser Woche nur am Donnerstag und Freitag gearbeitet. Der Student hat begonnen das Modul für einen LCD 1602 Bildschirm zu schreiben. Der Bildschirm hatte bei der Inbetriebnahme eine verschobene Zeile. Der Grund ist unbekannt.

Nächste Schritte:

- Grund für Bildschirmfehler eruieren

8.2 Zusammenfassung KW28

Ausgeführte Tätigkeiten:

Privater Bildschirm von zuhause zu Testzwecken mitgebracht. Der erste Bildschirm war defekt, eine Zeile war um zwei Pixel nach unten verschoben. Mit dem funktionierenden Bildschirm hat die Software sofort funktioniert.

In ETP-Testprogramme eingelesen. Das sind Skripte die auf Geräten von Kunden zum Testen ausgeführt werden. Das Verständnis dafür ist für die Fehleranalyse essenziell.

Der Softwareänderung für den ETP-Testadapter gewidmet. Programm wird nicht übersetzt da PROGMEM Variablen als Konstant qualifiziert werden müssen.

Nächste Schritte:

- Weitere Analyse der ETP-Testadapter Software

8.3 Zusammenfassung KW29

Ausgeführte Tätigkeiten:

Software des Testadapters weiter analysiert und verschiedene Tests mit dem Gerät durchgeführt.⁷

Erstellung des Docker Containers für die Entwicklung begonnen. Einpflegung des Projekts nach Bitbucket.

Nächste Schritte:

- Rollout Softwareupdate
- Erstellung Docker Container für STM32 und ETP-Testadapter Entwicklung

8.4 Zusammenfassung KW30

Ausgeführte Tätigkeiten:

Softwareänderung ETP-Testadapter ausgerollt und Geräte geflasht. Bei Ersatztestboxen wurde eine Einzeltestung durchgeführt, um die Funktionsfähigkeit zu garantieren.

Docker Container zum Übersetzen von alten Programmen begonnen aufzusetzen. Nach anfänglichen Schwierigkeiten und Abhängigkeitsproblemen eine Linux Version gefunden mit der der Compiler betrieben werden kann. Übersetzen von Software funktioniert, allerdings muss zurzeit jedes Mal der Quellcode in den Container kopiert, dort übersetzt und anschließend wieder zurück kopiert werden. Lösung dafür wird noch gesucht.

Docker Container mit PIO für die Entwicklung von STM32 Programmen begonnen aufzusetzen und zu testen. Bei Neuerstellung eines Projekts im Container stimmen die Pfade nicht. Es wird der falsche Slash-Typ verwendet.

Nächste Schritte:

- Erstellung Docker Container für STM32 und ETP-Testadapter Entwicklung
- Übungsaufgaben STM32 weitermachen

8.5 Zusammenfassung KW31

Ausgeführte Tätigkeiten:

PlatformIO Plugin im Docker Container getestet. Wenn das Projekt im Container erstellt wird, muss es per Command Line von einem Ort zum anderen kopiert werden, da ansonsten die

⁷ Interessenswahrung

Slash Richtung nicht stimmt. Github Ticket erstellt zur Fehlerbeschreibung und etwaigen Behebung.

Taktgeberübungen programmiert.

ETP-Testadapter Softwareänderung und Erkenntnisse dokumentiert.⁸

Nächste Schritte:

- ETP-Testadapter weiter dokumentieren
- Übungsaufgaben STM32 weitermachen

8.6 Zusammenfassung KW32

Ausgeführte Tätigkeiten:

In Azure eingelesen und mit der Weboberfläche gearbeitet.

Mit Benjamin Hartmann Probleme beim Terraformskript angesehen. Es gab dabei noch ein Problem, weil das Skript auf einem Windows Rechner geschrieben wurde und das Zeilenende dadurch auf CR/LF und nicht LF war. Skripte angepasst.

Systick-Übung weiterprogrammiert. Dabei fiel auf, dass die Informationen für den NVIC in einem anderen Handbuch stehen als Anfangs gedacht. Mit Michael Woog die Übung besprochen.

Dokumentation für ETP-Testadapter Software auf Stand gebracht.

Nächste Schritte:

- in Terraform einlesen und Tutorial machen
- Übungsaufgaben STM32 weitermachen

8.7 Zusammenfassung KW33

Ausgeführte Tätigkeiten:

In Terraform eingelesen. Terraform ist eine Sprache die „Infrastructure as code“ beschreibt. Damit können bei einem Webservice wie Azure Ressourcen dynamisch gemietet werden. Zudem wurde das Tutorial für die Anwendung mit Azure durchgemacht.

Für die CI/CD Phase 2 Vor- und Nachteile der einzelnen IDEs gesammelt.

⁸ Interessenswahrung

Bei der Taktgeberübung Problematik mit der Clock-Umstellung während der Laufzeit erkannt. Wenn die PLL als System Clock Source eingestellt ist und eine andere PLL-Konfiguration eingestellt werden soll, so muss als erstes die PLL abgeschaltet werden. Wenn dies passiert, hat der Prozessor keinen Clock mehr als Quelle.

Nächste Schritte:

- Übungsaufgaben STM32 weitermachen
- Thema für Bachelorarbeit finden und definieren

8.8 Zusammenfassung KW34

Ausgeführte Tätigkeiten:

Taktgeberübung weiter programmiert. Problem mit Clock-Wechsel On-The-Fly aufgetaucht. Wenn PLL als Clock-Quelle dient kann nicht einfach eine andere PLL-Konfiguration eingestellt werden. Zuvor muss die PLL als Quelle deaktiviert werden, eine neue Konfiguration eingestellt und dann PLL wieder als Clock-Quelle eingestellt werden.

LCD 1602 Modul geschrieben und getestet. Funktion zur Darstellung von Benutzerdefinierten Zeichen geschrieben und getestet.

DHT11 Modul geschrieben und getestet. Modul funktioniert noch nicht. Die Übungsaufgabe wurde in dieser BPP aus Zeitgründen nicht weiterverfolgt.

Thema für Bachelorarbeit gefunden und vorläufig fixiert. Es soll ein modularer Nachfolger des ETP-Testadapters werden.

Nächste Schritte:

- Übungsaufgaben STM32 weitermachen
- Besprechung mit Matthias Kolb bezüglich Bachelorarbeit
- Erstellung BPB2

8.9 Zusammenfassung KW35

Ausgeführte Tätigkeiten:

Besprechung zum Bachelorprojekt mit Matthias Kolb durchgeführt und Ideen festgehalten.

Betriebspraxisbericht erstellt.

Nächste Schritte:

- Betriebspraxisbericht abschließen

- Übergabe offener Arbeiten

8.10 Zusammenfassung KW36

Ausgeführte Tätigkeiten:

Betriebspraxisbericht fertiggestellt.

Arbeitsplatz aufgeräumt.

Nächste Schritte:

- Dispopapier BPP3 erstellen

9 Zusammenfassung der Abwesenheiten

Der Student war an folgenden Arbeitstagen Abwesend:

- vom 03.07.2023 – 05.07.2023
- vom 06.09.2023 – 28.09.2023

Durch den Urlaub am Ende der BPP konnten einige Punkte nicht mehr erledigt werden. Es wird aber dennoch davon ausgegangen, dass der Student sich die meisten der vereinbarten Kompetenzen des Dispopapiers aneignen konnte.

Anhang

Lerntagebuch BPP2

Interne Bewertung des Betreuers in der SIE

Lerntagebuch Betriebspraxisphase 2 vom 01.07.2023 – 30.09.2023

Name Student:

Michael Geuze

Name und Anschrift Firma:

System Industrie Electronic

Millennium Park 12

6890 Lustenau

Anschrift Betriebsstätte

(nur wenn Firma verschieden)

Kontaktdaten Duale/r Betreuer/in:

Name: Michael Woog

Kontaktdaten FHV-Betreuer/in:

Name: Patrick Ritschel

Telefonnummer (vorzugsweise Mobiltelefon) und E-Mail-Adresse:

T: +43 5577 89 90 0 - 287

E: michael.woog@sie.at

T: +43 5572 7923004

E: patrick.ritschel@fhv.at

Kontaktdaten Fachbetreuer/in:

Name: Martin Tomasini

Telefonnummer (vorzugsweise Mobiltelefon) und E-Mail-Adresse:

T: +43 5577 89 90 0 - 291

E: martin.tomasini@sie.at

Beginn – Ende Betriebspraxisphase:

01.07.2023 – 30.09.2023

Anmerkungen:

- Das Lerntagebuch ist tagesaktuell zu halten!
- Weitere Seiten sind nach Bedarf hinzufügen.

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
27	06.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Bezug Arbeitsplatz, Arbeitsmittel und PC eingerichtet, LCD-Modul für STM32 Controller begonnen zu schreiben
27	07.07.2023	Software-Engineering-Team (SWET)	Michael Woog	LCD-Modul für STM32 Controller mit 2 Zeilen Bildschirm getestet und debuggt; es sieht aus als wäre die untere Zeile verrutscht; keine Idee woher der Fehler kommen könnte;
28	10.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Eigener Bildschirm von zuhause mitgenommen und getestet; LCD in der Firma war defekt, Programm OK; bisher kann Text dargestellt werden; in ETP-Programme eingelesen und über den Prozess schlau gemacht;
28	11.07.2023	Software-Engineering-Team (SWET)	Michael Woog	ETP Testadapter Programm weiter eingelesen; [REDACTED] [REDACTED] [REDACTED] [REDACTED]
28	12.07.2023	Software-Engineering-Team (SWET)	Michael Woog	[REDACTED] [REDACTED] [REDACTED]

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
28	13.07.2023	Software-Engineering-Team (SWET)	Michael Woog	[REDACTED]
28	14.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Weitere Tests durchgeführt siehe Confluence; Programmänderung eingespielt und weiter getestet;
29	17.07.2023	Software-Engineering-Team (SWET)	Michael Woog	[REDACTED]
29	18.07.2023	Software-Engineering-Team (SWET)	Michael Woog	[REDACTED] weiter Debug-Ausgaben programmiert und Makefile angepasst; Programm neu übersetzt und geflasht;
29	19.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Weiterhin getestet; [REDACTED] [REDACTED]

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
29	20.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Meetings mit Michael Woog und Martin Tomasini; weiteres Vorgehen bestimmt; begonnen die flache Ordnerstruktur herzustellen; Sources und Includes angepasst; kompiliert; Docker Migration begonnen zu erstellen;
29	21.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Migration des Ursprungsprojektes nach Bitbucket; mit Bitbucket/Git vertraut gemacht; Readme's erstellt, begonnen Dokumentation SW Änderung zu fixieren; flashen der Controller; Docker Umgebung getestet; Binary kommt mit einer anderen Größe heraus; daher AVR-GCC Binary genommen; Validierung des SW-Updates abgeschlossen;
30	24.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Rollout des Softwareupdates an alle Controller; Kalibrierung aller Controller; Readme's erweitert
30	25.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Rollout des Softwareupdates an alle Controller; Kalibrierung aller Controller; Readme's erweitert
30	26.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Rollout des Softwareupdates an alle Controller; Ersatztestboxen separate Prüfung begonnen; Rollout abgeschlossen; Docker Integration für Build-Prozess versucht zu konfigurieren; Aufgrund des alten avr-gcc Compilers kann der Container nicht gebaut werden; Übergangslösung mit shell Skript und Makefile Execution und anschließender Rückspeicherung auf Host OS implementiert; Begonnen DevContainer für STM32 Framework in PIO vorzubereiten;

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
30	27.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Einzeltest für ETP-Testadapter weiter durchgeführt; Python3 Umgebungsvariablen in Windows angepasst; waren von WSL2/MINGW64 mehrere Versionen installiert; versucht bash Skript zur Docker-Erstellung/automatischen Kompilierung nach Python zu migrieren, bisher mit mäßigem Erfolg; es werden irgendwie mehrere Container instanziiert; das Kompilieren und zurückspeichern funktioniert allerdings -> Fehler war, dass die Option „intermediate container“ behalten aktiv war, daher die ganzen Images und Container; Problem mit WSL2 behoben, Python deinstalliert, mit Windows Paketmanager installiert; dann WSL neu installiert; dann wsl standard distribution wieder Ubuntu ausgewählt, war Docker-image -> dadurch konnte VSCode nicht mehr richtig starten; Begonnen Revision für ETP-Testadapter zu planen bzw. Brainstormen
30	28.07.2023	Software-Engineering-Team (SWET)	Michael Woog	Weiters die ETP-Testadapter getestet; versucht DevContainer für STM32 /PlatformIO zu konfigurieren; wenn ein neues Projekt angelegt wird, ist die Ordnerstruktur inkonsistent wegen den Slashes; es gibt im Linux Container auf einmal „/root\pro\test/“ und „/root/“, sprich es wird root/pro als Ordnername angesehen; noch keine Lösung; Versucht Hostlaufwerk zu mounten, um Fehler zu umgehen, geht noch nicht;
31	31.07.2023	Software-Engineering-Team (SWET)	Michael Woog	PlatformIO Community Post abgesetzt, laut Rückmeldung ein Ticket auf Github erstellt; scheint Bug von PIO zu sein; STM32 Schulung mit WOMI weitergemacht; Besprechung CI/CD Sicherheit teilgenommen;
31	01.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeber Übung weiter programmiert; Mirjam Huber bei PIO und Docker unterstützt; [REDACTED] [REDACTED] [REDACTED]

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
31	02.08.2023	Software-Engineering-Team (SWET)	Michael Woog	STM32 Paket für Martin Rabensteiner gepackt; Übung Taktgeber weiter programmiert; es gibt nun Funktionen mit denen SYSCLK, APB1, APB2, AHB1 und PLLCLK komplett eingestellt werden können; Verschönerung und Modularisierung fehlen noch; Logs von WinXP und DUT abgelegt und Dokumentation für ETP fertiggestellt
31	03.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeberübung weiter programmiert; Punkte zur Übung noch mit WOMI besprochen; Erweiterungsübung für unabhängige Konfiguration von MCO und SYSCLK mit WOMI festgelegt; Noch ein einzelner Testadapter ist aufgetaucht, Equipment wiedergeholt und geflasht etc.; in Terraform eingelesen, Tutorial begonnen;
32	07.08.2023	Software-Engineering-Team (SWET)	Michael Woog	
32	08.08.2023	Software-Engineering-Team (SWET)	Michael Woog	etwas in Azure Cloud-Dienste eingearbeitet; Budgetbedenken mit IT abgesprochen; Terraform Probleme mit Benjamin Hartmann angesehen; Shells waren CRLF und nicht LF -> mit dos2win Implementierung behoben; Problem in Skript behoben (Skript wurden als Dateien und nicht als Skript ausgeführt); Terraform Skript geht nun;
32	09.08.2023	Software-Engineering-Team (SWET)	Michael Woog	ETP Testadapter Rollout der Firmwareänderung auf V1.31 mit den LTT-Testadaptern durchgeführt; halber Tag Urlaub;

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
32	10.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Einer Probearbeitenden meine Arbeiten gezeigt; Rollout SW Update weitergeführt; Bitbucket Projekt auf neuen Stand gebracht; Confluence Dokumentation auf Stand gebracht; Taktgeberübung Systick versucht zu verstehen; kann nur in der HAL nachforschen und nachsehen, wie das funktionieren sollte; keine Ahnung wie das SYSTICK Register laut Hersteller angesteuert werden sollte;
32	11.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeberübung Systick mit WOMI besprochen -> das betreffende Register steht in der Programming Manual und nicht im Reference Manual; Bitbucket Vorstellung am Montag vorbereitet; STMCubeIDE Probleme behoben;
33	14.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Besprechungen: Ausbildungstreff(2h), CI/CD Fragerunde(1h), Terraform DevContainer Anwendungsvorstellung(1,5h), SWE Team Meeting(1,5h)
33	15.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Feiertag
33	16.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Vor- und Nachteile der einzelnen IDEs gesammelt für die spätere Entscheidung ; VPN und VPN Tunnel Lehrvideos angesehen; begonnen alle STM-Projekte auf Cube V1.13 zu ändern

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
33	17.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeberübung weitergemacht und analysiert; Lösung ist für mich nicht leicht verständlich -> werde mit Michael Woog reden; Problem besprochen, Lösungsdokumentation erweitert
33	18.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeberübung weitergemacht, Problem mit Clock-Umstellung während der Laufzeit erkannt, muss noch geändert werden
34	21.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Taktgeberübung mit Clockumstellung on the fly fertiggestellt; aufpassen, wenn die derzeitige Clocksource PLL ist und abgeschaltet werden muss; zuerst dann auf eine Source wechseln welche noch aktiv ist und dann auf die geänderte PLL-Quelle zurückgehen; Begonnen, LCD-Bildschirmmodul zu schreiben; war schon ziemlich weit vom Juli; einige Funktionen noch implementiert und generalisiert;
34	22.08.2023	Software-Engineering-Team (SWET)	Michael Woog	LCD-Modul fertiggestellt; Special Characters implementiert; Fehlerhaftes Verhalten des Mikrocontrollers festgestellt; wenn der nicht isolierte STLink verwendet wird, kann es vorkommen, dass der STM noch Spannung über den Bildschirm bekommt, dadurch gibt es flackernde Signale, STLink wird z.T. nicht mehr erkannt; Controller wird dann auch nicht zu mehr komplett neu gestartet
34	23.08.2023	Software-Engineering-Team (SWET)	Michael Woog	DHT11 Modul geschrieben, und getestet -> funktioniert noch nicht; Sammlung für ETP Testadapter Revision/Bachelorarbeit erstellt -> heißt jetzt FlexiBox;

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
34	24.08.2023	Software-Engineering-Team (SWET)	Michael Woog	FlexiBox Sammlung erweitert; mit Matthias Kolb Änderungen besprochen und in Dokument eingepflegt; Übergabe ETP-Testadapter mit Michael Woog durchgeführt;
34	25.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Besprechung ursprüngliches Konzept für ETP-Automatisierung; Besprechung Bachelorarbeit Michael Geuze; Übergabe Urlaub Woog Michael;
35	28.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Besprechung mit Matthias Kolb wegen Bachelorarbeit und Prozessautomatisierung in der Produktion; Erstellung Übergabedokument Urlaub Michael Geuze, [REDACTED]
35	29.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Besprechung mit Martin Tomasini; Martin Rabensteiner Kennenlernen; Erstellung Betriebspraxisbericht (BPB) 2
35	30.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Besprechung mit Martin Tomasini, Erstellung BPB2
35	31.08.2023	Software-Engineering-Team (SWET)	Michael Woog	Erstellung BPB2

Woche	Tag	Abteilung	Betreuer/in	Tätigkeiten
35	01.09.2023	Software-Engineering-Team (SWET)	Michael Woog	Erstellung BPB2
36	04.09.2023	Software-Engineering-Team (SWET)	Michael Woog	Erstellung BPB2
36	05.09.2023	Software-Engineering-Team (SWET)	Michael Woog	Aufräumen Arbeitsplatz, letzter Arbeitstag vor Urlaub

Beurteilung durch den / die Dualbetreuer/in im Partnerunternehmen

Name des/der Student/in	Geuze			
Vorname des/der Student/in	Michael			
Betriebspraxisphase		1		
	X	2		
Praxisvertiefung		Betriebswirtschaft	X	Elektronik
	X	Elektrotechnik	X	Informatik
Zeitraum	01.07.2023 – 30.09.2023			
Partnerunternehmen	S.I.E. Headquarters System Industrie Electronic GmbH Millennium Park 12 AT-6890 Lustenau			
Betriebsstätte	MP12			
Dualbetreuer	Michael Woog			

Aufgaben und Tätigkeiten, Ausbildungsschwerpunkte

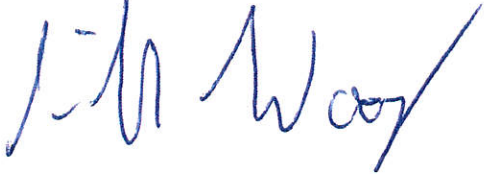
- Einarbeitung in CI/CD Infrastruktur
 - Kennenlernen der Tools wie im Bericht beschrieben
 - Verständnis für den Ablauf der Standardbearbeitung eines Änderungsauftrags
 - Erstellung von Docker Containern mit verschiedenen Aufgaben
- Softwareänderung für den ETP-Testadapter¹
- Programmieren mit dem STM32 Mikrocontroller
 - Einstellen der Clocks mittels Bare-Metal-Programmierung

Zusammenfassende verbale Beurteilung:

Die Ausarbeitung seiner übertragenen Aufgaben löste Herr Geuze selbständig, gewissenhaft und in kurzer Zeit. Die von ihm gewonnen Erkenntnisse und Ausarbeitungen waren ein wichtiger Beitrag zu den Projektfertigstellungen.

Gerne teile er sein Wissen mit seinen Kollegen und Studenten.

¹ Um die Interessen der Firma SIE und deren Kunden zu wahren können Details dieser Arbeit nicht veröffentlicht werden. Da möglicherweise erkennbar ist, um welches Produkt oder welchen Kunden es sich handelt werden Technische Lösungen und Tätigkeiten nur oberflächlich beschrieben und teilweise ganz weggelassen. Im Rahmen der Beurteilung ist eine Einsicht nach Absprache mit der Firma SIE möglich.

Ort und Datum: Lustenau, den 05.10.2023
Unterschrift des/der Dualbetreuer/in 
Firmenstempel  system industrie electronic gmbh System Industrie Electronic GmbH Millennium Park 12 A-8890 Lustenau Tel. +43(0)77/60000-0, Fax 35901

Arbeitsqualität		1	2	3	4
Analytische und konzeptionelle Fähigkeiten	Erarbeitet sinnvolle Lösungsansätze	X			
Lernbereitschaft	Nimmt angebotene Lernmöglichkeiten an und nutzt diese	X			
Kreativität	Bringt neue Denkmuster und/oder Arbeitsmethoden ein		X		
Arbeitsweise		1	2	3	4
Zuverlässigkeit	Erfüllt Aufgaben verantwortungsvoll und sorgfältig	X			
Effizienz	Organisiert die Arbeit sinnvoll und vermeidet unnötige Arbeitsschritte	X			
Flexibilität	Stellt sich problemlos auf neue/veränderte Aufgabenstellungen ein	X			
Motivation und Selbststeuerung		1	2	3	4
Engagement	Zeigt Interesse und Einsatzbereitschaft	X			
Zielorientierung	Setzt Aufgaben in realistische Ziele und verfolgt diese	X			

Bericht für die Betriebspraxisphase 2 vom 01.07.2023 – 30.09.2023

Belastbarkeit	Bewältigt schwierige Arbeitsbedingungen und/oder hält hohem Arbeitsdruck stand	X			
Kritikfähigkeit / Umsetzungsvermögen	Kann mit positiven und negativen Rückmeldungen umgehen, hinterfragt eigenes Verhalten	X			
Teamfähigkeit		1	2	3	4
Integrationsfähigkeit	Integriert sich ins Arbeitsumfeld und findet Akzeptanz unter den Kolleg/innen und Vorgesetzt/innen	X			
Zusammenarbeit / interkulturelle Kompetenz	Tritt verlässlich und angemessen auf	X			
Argumentationsvermögen	Argumentiert und spricht schlüssig und klar	X			
Dialog- und Konfliktfähigkeit	Spricht Probleme offen an, lässt andere Meinungen zu, hat Mut zur konstruktiven Auseinandersetzung	X			

Detaillierte Beurteilung: 1 = sehr stark vorhanden, 2 = stark vorhanden, 3 = vorhanden, 4 = kaum wahrnehmbar (wird als Lernfeld empfohlen)